# Doubly Linked List

A doubly linked list is a list which allows us to travel to the next node or the previous node from the current node. To facilitate this, every node apart from the component has two pointers – one to the next normally called next and the other called prev for previous.

The interface of a doubly linked list is same as that of a singly linked list.

It supports the following operations.

- Initialize the list
- Deinitalize the list
- Insert anywhere in the list
- Delete anywhere in the list
- Traverse
- Search for a key

The doubly linked list structure will have two pointers:

head_ : this will point to the first node.

tail_ : this will point to the last node.

```
struct DLL {
        node_t *head_;
        node_t *tail_;
};
typedef struct DLL dll_t;
```

The node structure will also have two pointers – traditionally called prev and next.

```
struct node
{
   int key_;
   struct node *prev_;
   struct node *next_;
};
typedef struct node node_t;
```

- **Initialize the list**
  The head and the tail of the doubly linked list are grounded.
- **Deinitalize the list**
  This requires a traversal of the routine and freeing of each node. We should always have a pointer to the next node before freeing the current node. Otherwise, the remaining list will become garbage and therefore memory leak.
- **Insert anywhere in the list**
  In case of a singly linked list, we had to have two pointers, one following the other as we could not move back in the list. That restriction no more exists in a doubly linked list. To insert a node temp before the current node pres, we could do the following changes.
  temp->next_ = pres;  temp->prev_ = pres->prev_;
  pres->prev_->next_ = temp; pres->prev_ = temp;
  You may observe that we make 4 changes while inserting.

  The above code works only if we are inserting in the middle. What if we insert in
  - An empty list : both head_ and tail_ of the DLL should be changed
  - In the beginning : head_ of the DLL has to change
  - At the end : tail_ of the DLL has to change

- **Delete anywhere in the list**

  We should consider the following cases.

  - Deletion in the middle
  - Deletion of the first node – head_ of DLL has to change
  - Deletion of the last node – tail_ of DLL has to change
  - Deletion of only one node – both head_ and tail_ of DLL have to change.

  What do you think this code will do if temp point a node in the middle of the list?

  temp->next_->prev_ = temp->prev_;

  temp->prev_->next_ = temp->next_;;

  free(temp);

- **Traverse** :

  We can traverse forward or backward as we have links in both the directions.

---

**Why doubly linked lists?**

A singly linked list can do whatever a doubly linked list can do. Then, what do we gain by creating a doubly linked list?

As we can traverse in both directions, insertion, deletion become easier.

We use more space – extra pointer in each node. We gain in time.

This is a classical example of **Time Space Trade Off.**

Let us compare the performance of singly and doubly linked lists in terms of number of steps required for each of the following operations in the worst case. Assume that the list has n nodes.

- Move forward :  singly linked list : 1 step; doubly linked list : 1 step : no difference
- Move back : singly linked list : n step; doubly linked list : 1 step :
- Insert before a node : singly linked list : 1 step; doubly linked list : 1 step :
- Delete a node : singly linked list : 1 step; doubly linked list : 1 step :
- Searching for a key : singly linked list : n step; doubly linked list : n step :

**A doubly linked list does NOT make searching faster.**

The advantage is **bidirectional traversal and easier local insertion/deletion**.

**Applications of Doubly Linked List :**

**Browser History**

**Music Player Playlist**

**Undo and Redo operations**

**LRU cache**

**Round Robin Scheduling**